

嵌入式操作系统

Embedded Operating System

信息与软件工程学院

桑楠

sn@uestc.edu.cn

2026年4月

第五部分

实时调度策略

核心内容

- 相关概念
- 典型的实时调度策略
- RMS调度
- EDF调度
- 优先级反转及解决

相关概念

基本概念

- **任务**：相互作用的程序集合或软件实体，每个程序执行时成为任务。
- **任务执行的特性**：
 - **动态性**：任务的运行状态不断变化 → 状态变迁图
 - **并发性**：多任务并发执行
 - **异步独立性**：任务间相互独立，不存在前驱与后继关系
 - **同步性**：任务之间非独立，相互依赖
- **调度点的位置**：中断结束、等待资源、周期起止、...

基本概念 (2)

- **任务**：相互作用的程序集合或软件实体，每个程序执行时成为任务。
- **时间约束**：
 - **截止时间**：任务必须完成的时间
 - **执行周期**：任务重复出现并处理完成的时间间隔
 - **延迟时间**：任务允许等待的时间
- **RTOS追求的目标**：简单、确定性 → 确保实时性约束
- **通用调度是NP问题**；复杂调度往往只用于理论研究

调度机制与调度策略

- **调度机制**：根据给定调度策略来安排任务的具体执行，如创建任务、就绪任务、挂起任务、任务间通信、...。
- **调度策略**：针对有限的CPU资源，确定哪个任务先执行、在哪个CPU上执行、执行时间、等等的策略。→ 以调度算法的形式体现
- **调度策略的本质**：为任务确定优先级
- **调度算法**：在特定时刻，确定将要运行的任务的一组规则
- **调度机制和调度策略共同完成内核的调度功能**

调度策略/算法必须考虑的因素

- 截止时间：Deadline
- 任务响应时间：Response time
- 确定性：Predictability
- CPU和资源的利用率：Utilization
- I/O设备吞吐率：Throughput
- 公平性：Fairness
-

任务调度需要解决的问题

- **WHAT**: 按什么原则分配CPU
 - 任务调度算法
- **WHEN**: 何时分配CPU
 - 任务调度的时机
- **HOW**: 如何分配CPU
 - 任务调度过程（任务的上下文切换）

有关调度算法的研究

- **理论上**：最优调度只有在能够完全获知所有任务在处理、同步和通信方面的需求，以及硬件的处理和时间特性的基础上才能实现。
 - **实际应用**：很难，特别是信息需求
 - **结论**：即使条件满足，仍然难以实现
 - **调度的复杂性**：随任务和约束的数量指数增长
- **现状**：算法很难适应负载和资源不断增长的系统
- **应用**：适应特定的、定义好任务的系统的需求

类似或拓展的许多研究：
软件工程、覆盖测试、
大数据分析、.....

典型的 实时调度策略

基本分类

- **抢占式和非抢占式**：根据任务运行过程中能否被抢占。
- **离线调度和在线调度**：根据获得调度信息的时机
 - **离线调度**：在系统运行之前确定所有需要的调度信息，
具有很高的确定性，但不灵活
 - **在线调度**：系统运行时确定调度信息
- **静态调度和动态调度**：优先级预定或动态确定
- **最优调度和启发式调度**：根据能否确保系统性能最优 →
实际工程中，通常采用启发式调度

另一种分类

- **离线调度**：脱机配置产生所有运行时调度所需的静态信息。
- **运行时调度**：在系统运行时，根据发生的不同事件，在各个计算之间进行切换处理（**每个计算相当于一个任务**）
- **先验性分析**：根据静态信息和调度算法在运行时的行为分析，确定所有时间需求是否得到满足
- **三类调度之间的差异**：调度行为的侧重点不同 → **不变、只顾当前、整体考虑**

比较

- **静态调度和动态调度**：根据优先级确定的时机
 - **广义**：静态调度的调度策略在运行前确定，且调度不随当前环境变化而改变。如**轮询**、源地址哈希、...
 - 动态调度可以确保低优先级任务也能被调度
 - 需要**绝对可预测性**的系统一般不使用动态调度；
 - 所有任务都具有**同等重要程度**的系统可用动态调度
 - **动态调度的优先级只反应时间特性，未考虑紧迫性**
 - **动态调度的调度代价通常高于静态调度**

RMS

实时调度算法

来源

- **调度算法**：是在一个特定时刻用来确定将要运行的任务的一组规则
- **实时调度算法**：从1973年Liu和Layland开始关于实时调度算法的研究工作以来（1973年，Liu和Layland发表了一篇名为“Scheduling Algorithms for Multiprogramming in a Hard Real-Time Environment”的论文），相继出现了很多调度算法和方法
- **内容**：RMS、EDF、...

符号定义

- 系统中有 n 个实时任务 T_1 、 T_2 、...、 T_n
- 任务的周期分别为 P_1 、 P_2 、...、 P_n
- 任务的执行时间分别为 E_1 、 E_2 、...、 E_n
- 任务的相对截止时间分别为： D_1 、 D_2 、...、 D_n
- 任务的优先级分别为： Pri_1 、 Pri_2 、...、 Pri_n ，且 Pri_i
越小，优先级越高

速率单调调度算法：RMS

- **类型**：最好的静态优先级调度算法
- **假设**：任务集T有n个实时任务 T_1 、 T_2 、...、 T_n
 - 所有任务都是**周期任务**， P_1 、 P_2 、...、 P_n
 - **任务的周期越小，优先级越高**： $Pri_i = P_i \rightarrow$ **单调**
 - 任务的相对截止时间等于任务的周期： $D_i = P_i$
 - 任务在每个周期内的**计算时间** **周期越短？**
 - 任务之间相互独立：不进行通信，也不需要同步
 - 任务可以在计算的任何位置被抢占，不存在临界区

RMS：可调度性判定

- **系统可调度**：每个任务 T_i 都可在其截止时间 D_i 之前完成
- **判定**：任务集 T 可调度的充分条件是

$$\text{CPU的利用率} \leq n(2^{1/n} - 1)$$

其中 n 为任务个数

C. L. Liu, J. W. Layland, “*Scheduling Algorithms for Multiprogramming in a Hard Real-Time Environment*”,
ACM, 1973

RMS：执行示例1

任务	执行时间	周期时间
T_1	1	4
T_2	1	5
T_3	1	10

RMS：执行示例1 (2)

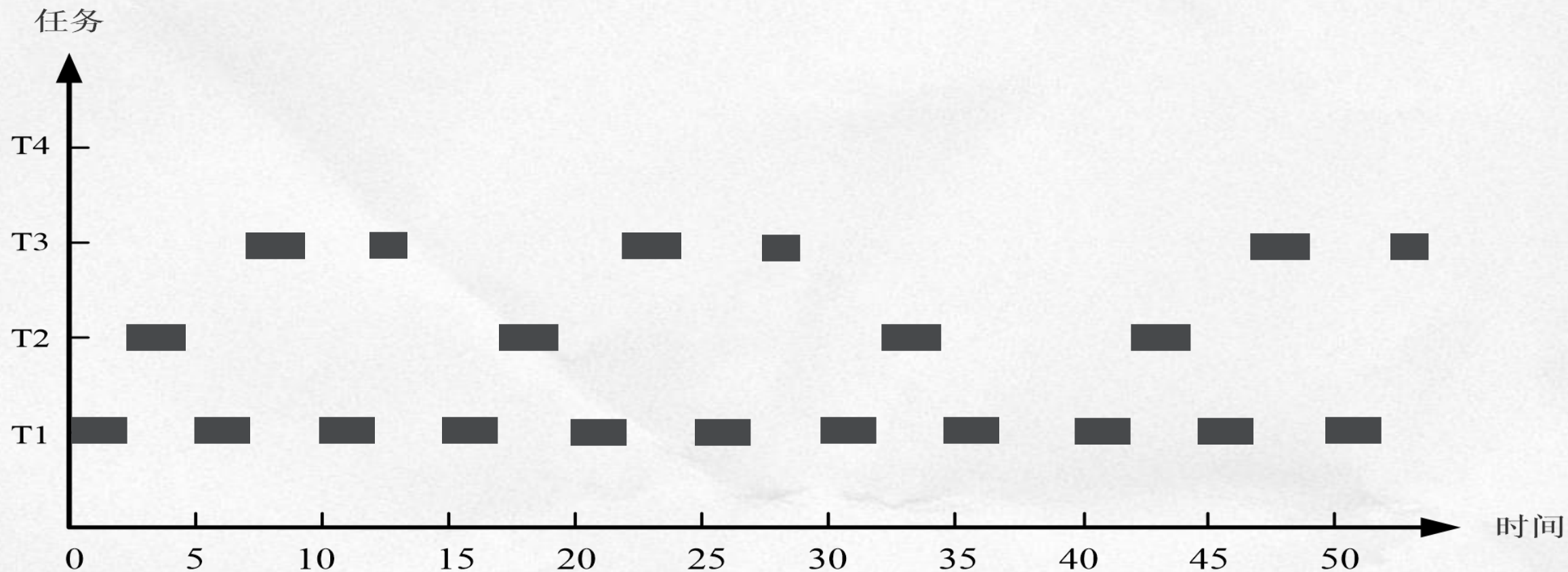
- CPU利用率： $\sum U_i$
- U_i ：第*i*个任务的CPU使用率
 - 假设：周期任务： $T_i(P_i, E_i)$ ，则： $U_i = E_i / P_i$
- 例1：任务集 $T = (T_1, T_2, T_3)$
 - $\sum U_i$ ： $1/4 + 1/5 + 1/10 = 0.55$
 - $n(2^{1/n}-1)$ ： $3(2^{1/3}-1) \approx 0.77976$
 - 结论：T可调度

RMS：执行示例2

任务	执行时间	周期时间
T_1	2	5
T_2	3	15
T_3	4	20

RMS：执行示例2 (2)

- 例2：初始时所有任务同时到达



T1、T2、
T3到达

RM算法

RMS：执行示例2 (3)

- 例2：任务集 $T = (T_1, T_2, T_3)$
 - $\sum U_i$: $2/5 + 3/15 + 4/20 = 0.8$
 - $n(2^{1/n}-1)$: $3(2^{1/3}-1) \approx 0.77976$
 - 结论：T可调度性条件不充分

RMS：可调度的CPU使用上限

任务数量	可调度的CPU使用率上限
1	1
2	0.828
3	0.780
4	0.757
5	0.743
6	0.735
...	...
∞	$\ln 2$

RMS：优缺点及其他

- **优点**：如果一组任务能够被任何静态调度算法所调度，则这些任务在RMS下也是可调度的
- **不足**：可调度的CPU使用率上限为 $\ln 2$ ，如此低的CPU使用率对大多数的系统来说不可接受
- **不足**：假定任务是相互独立的、周期性的，且能够在任何计算点被抢占、不竞争资源
- **非周期任务的处理方式**：后台方式执行非周期任务；周期性地查询非周期任务

EDF

实时调度算法

最早截止时间优先算法：EDF

- **类型**：最好的单处理器动态调度算法
- **假设**：任务集T有n个实时任务 T_1 、 T_2 、...、 T_n
 - 所有任务**不必都是周期任务**， P_1 、 P_2 、...、 P_n
 - 任务的相对截止时间等于任务的周期： $D_i = P_i$
 - 任务在每个周期内的计算时间都相等
 - 截止时间越短的任务具有越高的优先权
 - 任务之间相互独立：不进行通信，也不需要同步
 - 任务可以在计算的任何位置被抢占，不存在临界区

EDF：可调度性判定

- **系统可调度**：每个任务 T_i 都可在其截止时间 D_i 之前完成
- **判定**：任务集 T 可调度的充要条件是

$$\text{CPU的利用率} \leq 1$$

$$\text{即：} (C_1/T_1) + (C_2/T_2) + \dots + (C_n/T_n) \leq 1$$

其中 n 为任务个数

C. L. Liu, J. W. Layland, “*Scheduling Algorithms for Multiprogramming in a Hard Real-Time Environment*”,
ACM, 1973

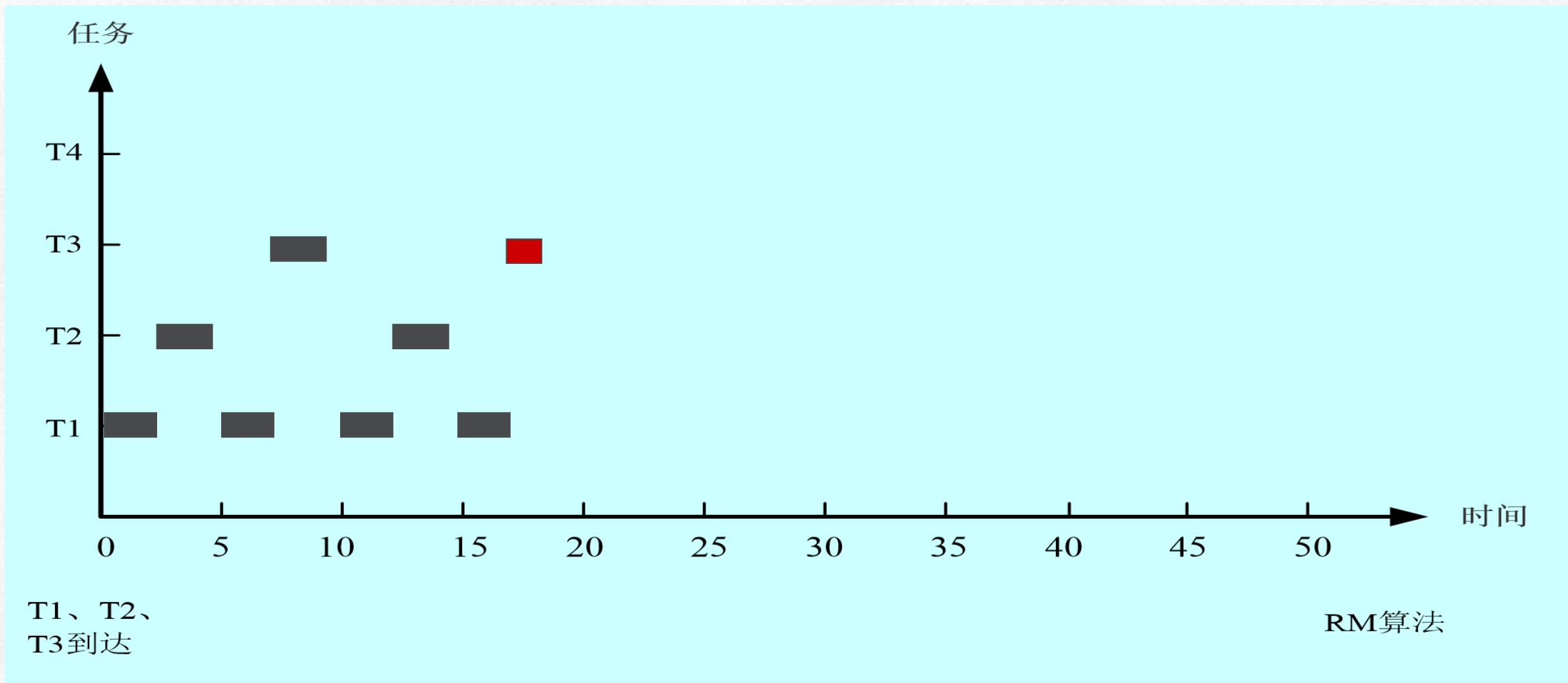
EDF：示例3

任务	执行时间	周期时间
T_1	2	5
T_2	3	10
T_3	4	15

RMS：执行示例3

- 例3：任务集 $T = (T_1, T_2, T_3)$
 - $\sum U_i$: $2/5 + 3/10 + 4/15 \approx 0.96667$
 - $n(2^{1/n}-1)$: $3(2^{1/3}-1) \approx 0.77976$
 - 结论：T可调度性条件不充分

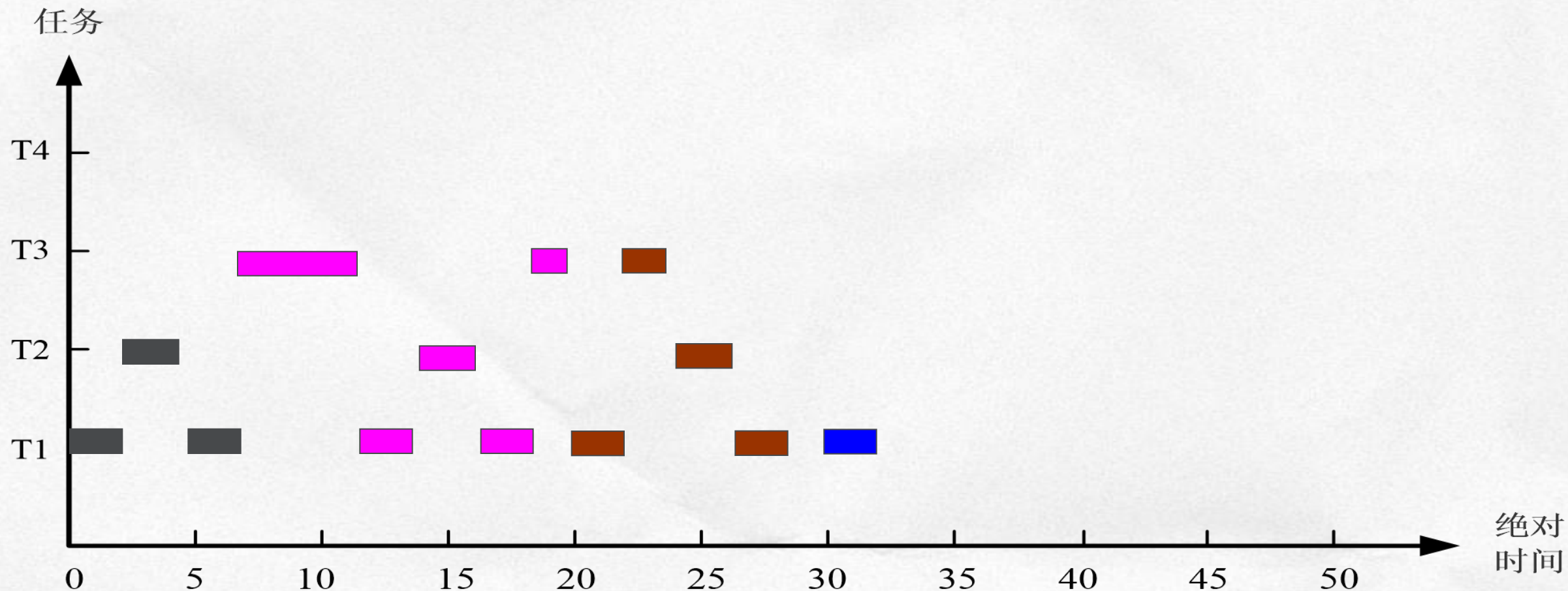
RMS：执行示例3 (2)



EDF：执行示例3

- 例3：任务集 $T = (T_1, T_2, T_3)$
 - $\sum U_i$: $2/5 + 3/15 + 4/20 \approx 0.96667$
 - $\sum U_i$: < 1
 - 结论：T可调度

EDF：执行示例3 (2)



T1、T2、
T3到达

EDF算法

EDF：优缺点及其他

- **优点**：可调度上限为100%，可充分利用CPU的计算能力
- **不足**：在实时系统中不容易实现
- **不足**：与RMS相比，EDF具有更大的调度开销
- **不足**：假定任务是相互独立的、周期性的，且能够在任何计算点被抢占、不竞争资源
- **不足**：系统临时过载时，不能确定哪个任务的截止时间得不到满足

LLF

实时调度算法

最小松弛度优先算法：LLF

- **类型**：最好的单处理器动态调度算法，Least-Laxity-First Scheduling
- **假设**：与EDF相同
 - 所有任务**不必都是周期任务**， P_1 、 P_2 、...、 P_n
 - 任务的相对截止时间等于任务的周期： $D_i = P_i$
 - 任务在每个周期内的计算时间都相等
 - 任务的松弛度（**空闲时间**， $D_i - E_i$ ）越短，任务的优先级越高
 - 任务之间相互独立：不进行通信，也不需要同步
 - 任务可以在计算的任何位置被抢占，不存在临界区

LLF：可调度性判定

- 任务的执行时间： E_i
- 任务的空闲时间： D_i - 任务剩余执行时间
- 判定：任务集T可调度的充要条件是

CPU的利用率 ≤ 1

即： $(C_1/T_1) + (C_2/T_2) + \dots + (C_n/T_n) \leq 1$

其中n为任务个数

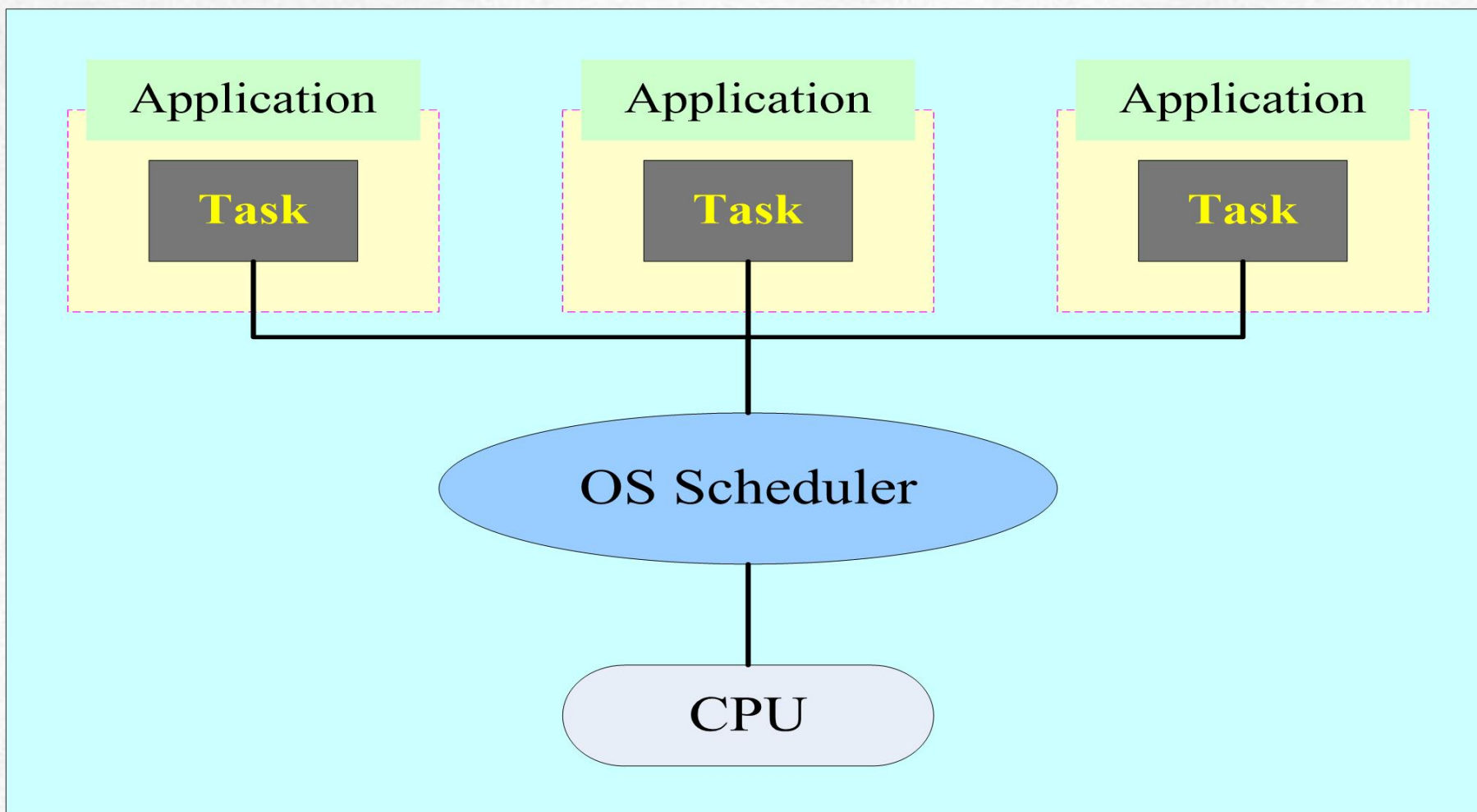
LLF：示例3

任务	执行时间	周期时间
T_1	2	5
T_2	3	10
T_3	4	15

实时调度算法 的一些研究

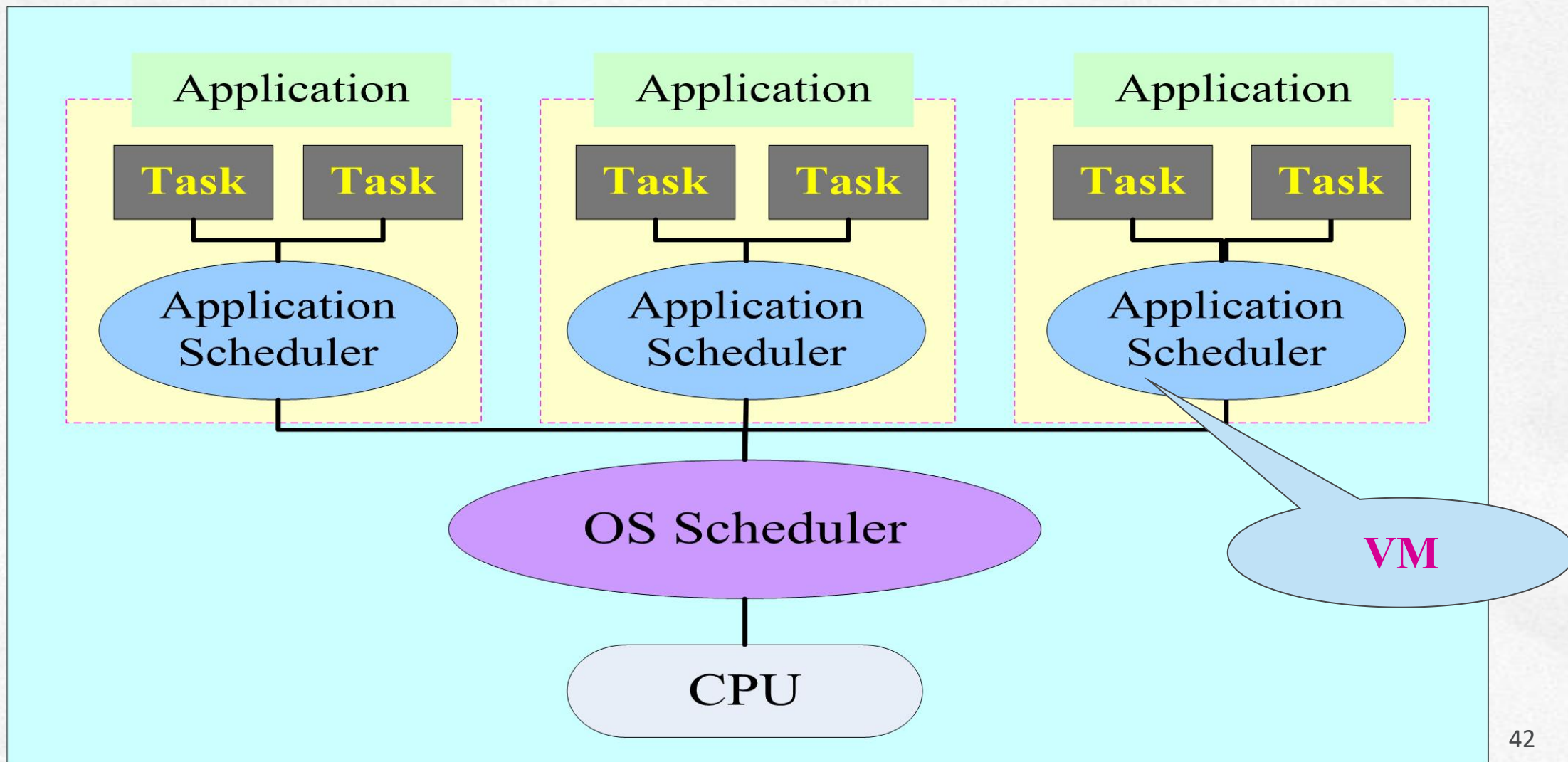
传统的调度框架

- 一个应用中只有一个任务的模式



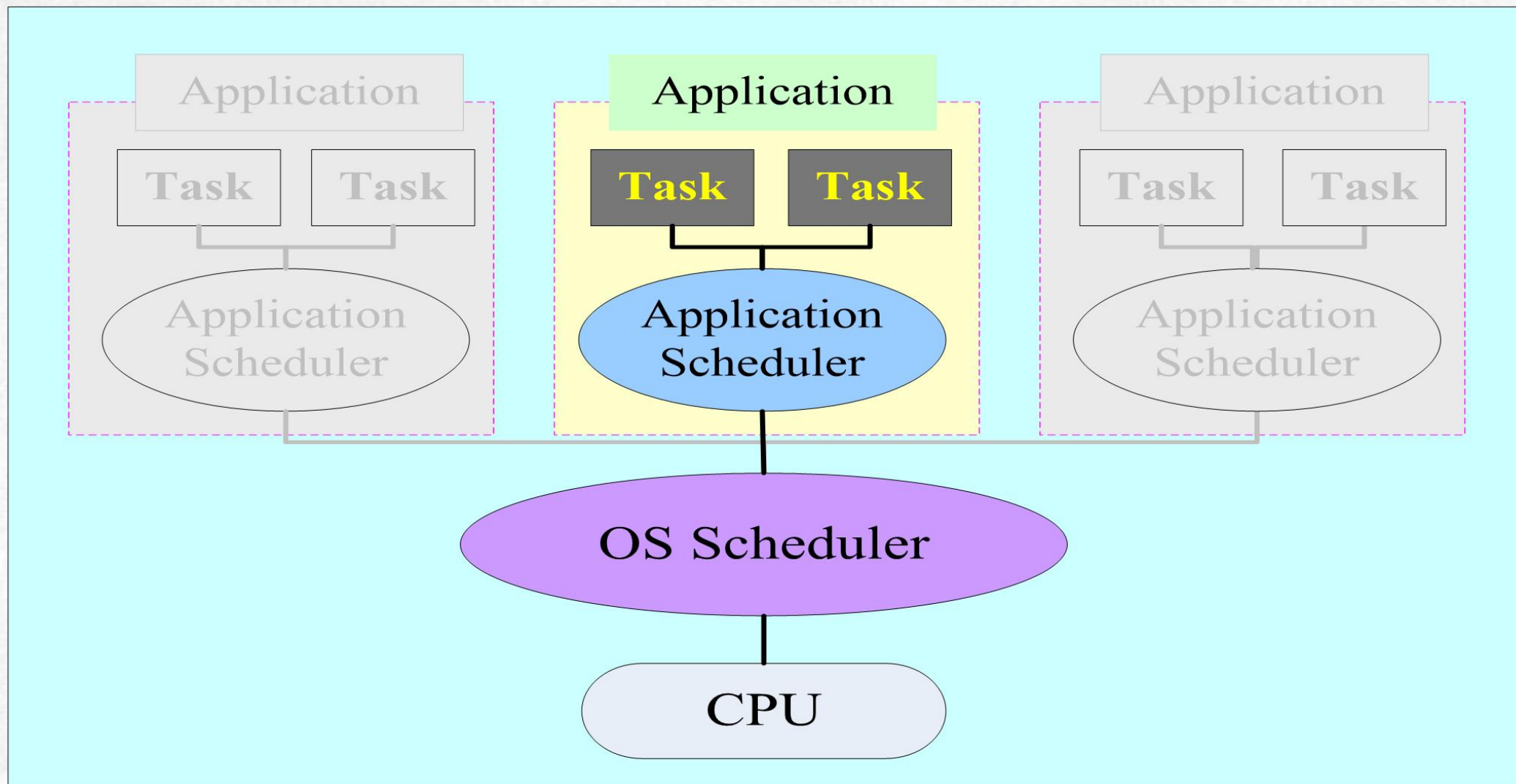
层次调度框架 (HSF)

- 一个应用中含有多个实时任务、且有一个调度器的模式



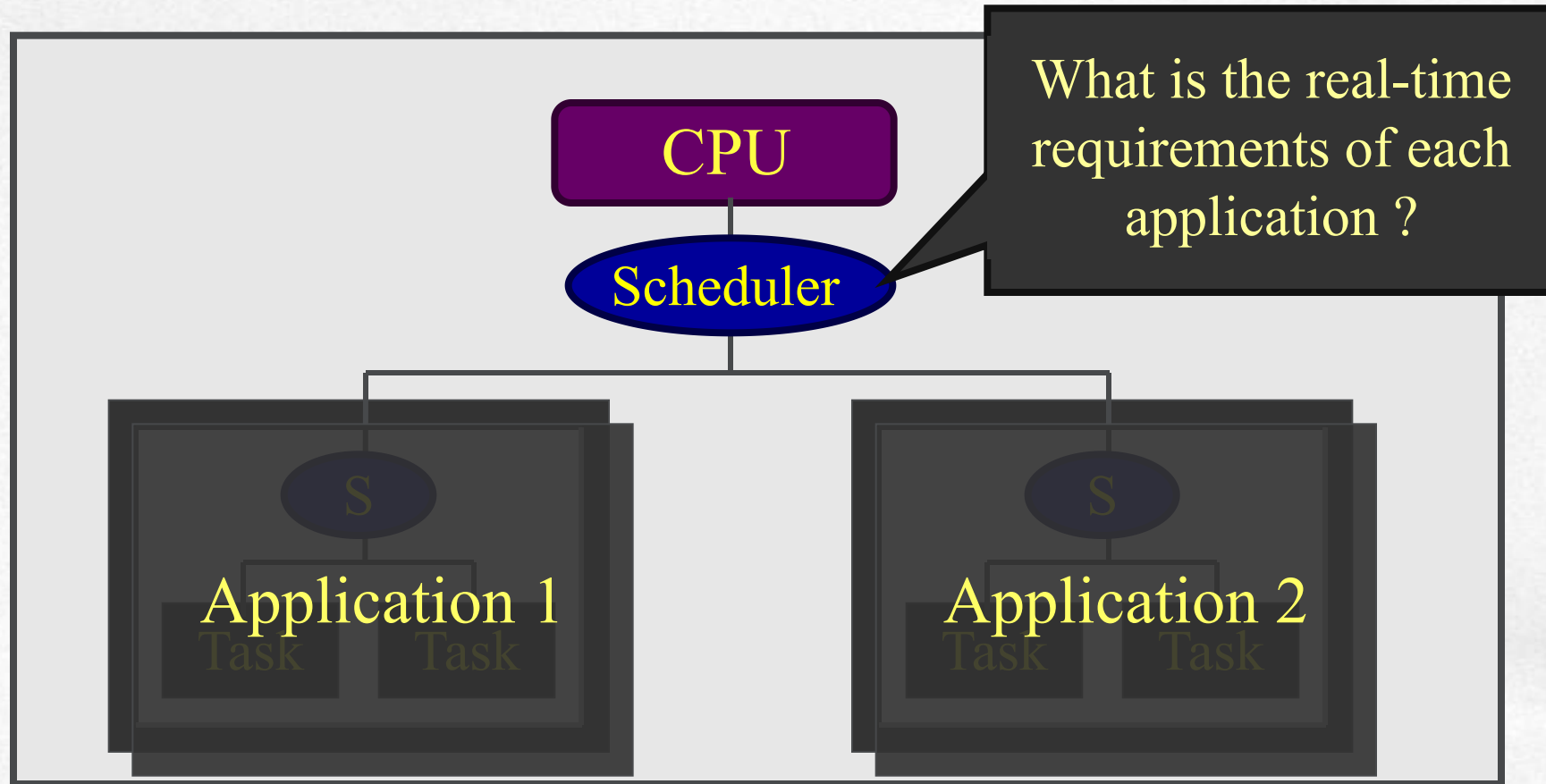
所期望的HSF调度性能

- 独立的可调度能力分析



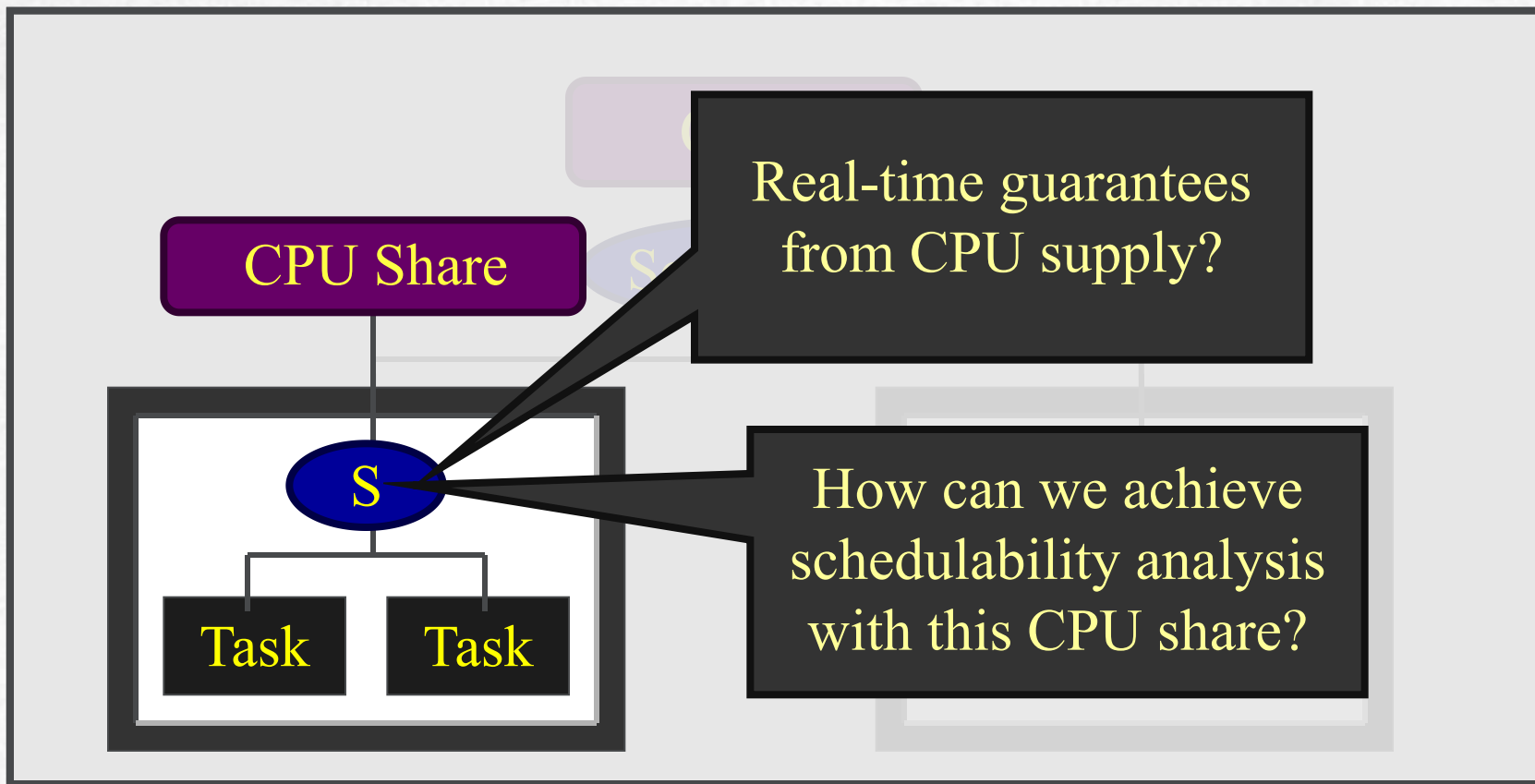
层次框架：一些研究成果

- 系统级观点



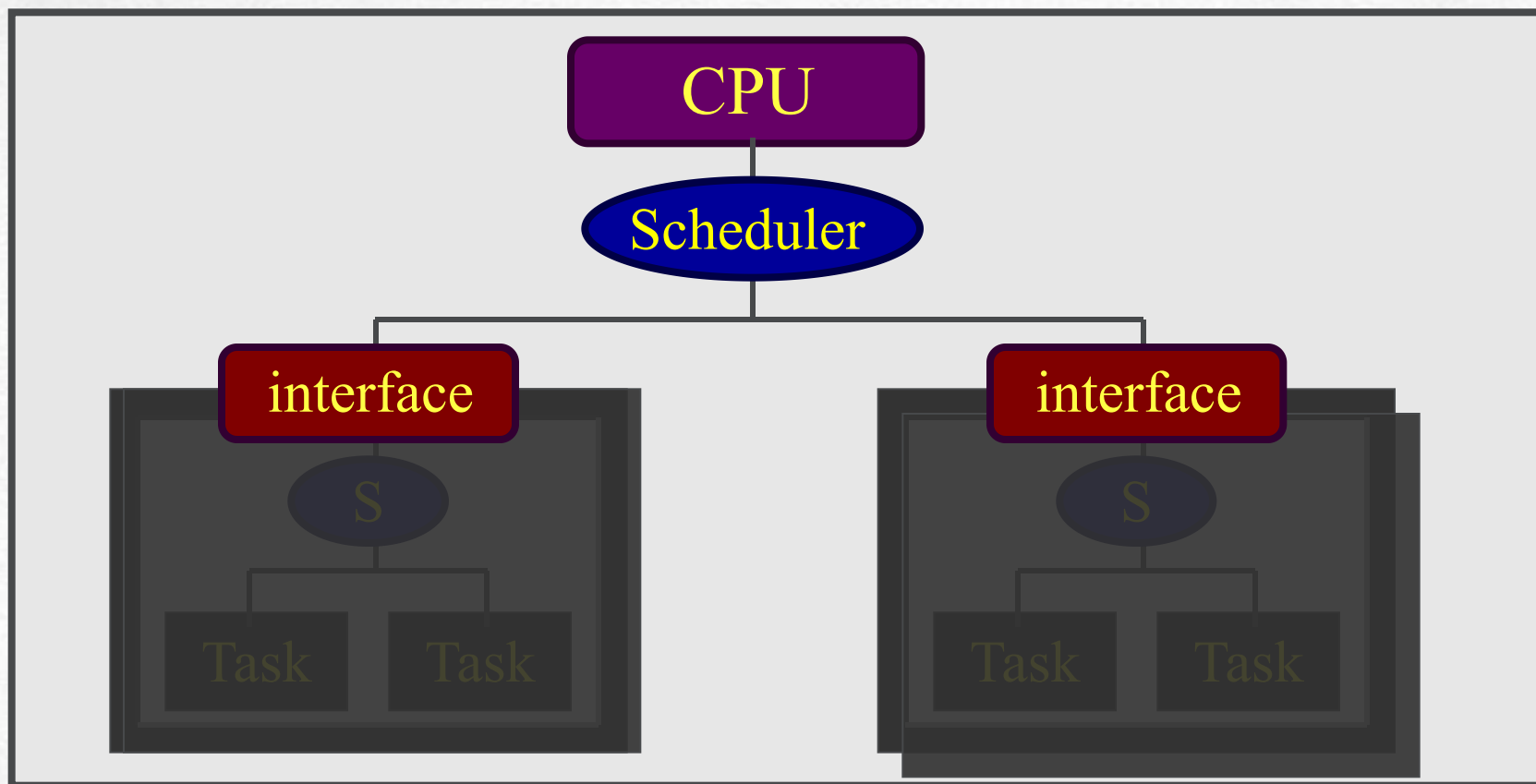
层次框架：一些研究成果 (2)

- 应用级观点



期望的框架：想法

- 基于接口的层次调度框架

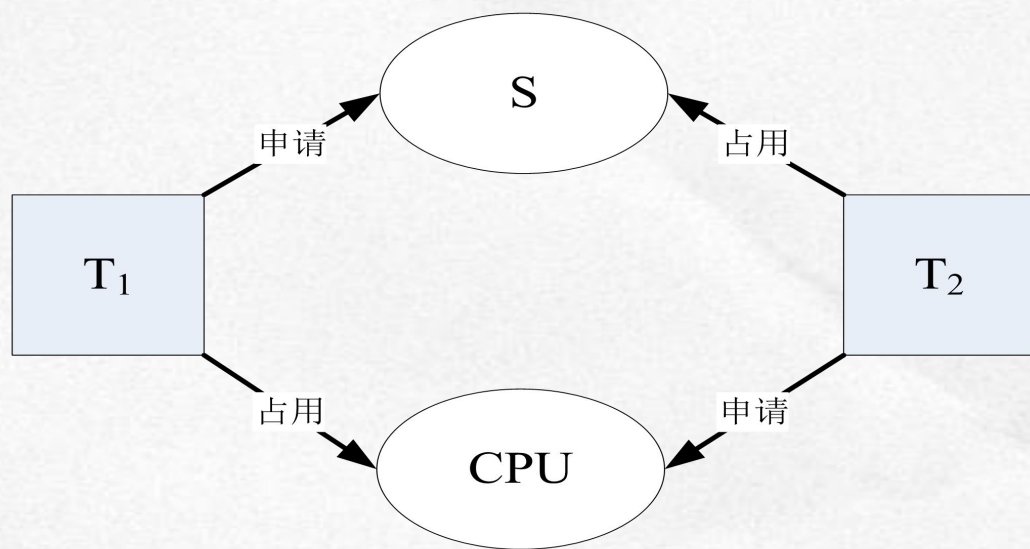


死锁与 优先级反转

死锁

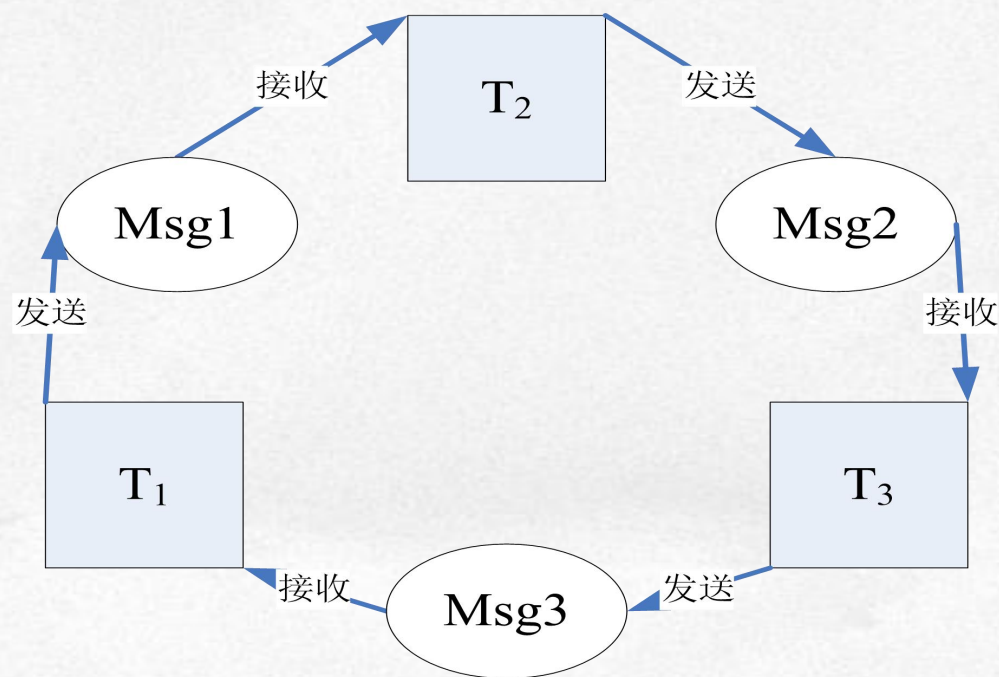
- **定义**：一组任务中，每个任务都无限等待该组中另一任务占用的资源，而永远的不到资源的现象

- **直接死锁**



直接死锁

- **间接死锁**



间接死锁

死锁的产生及预防

- **死锁产生**：需要满足四个条件
 - **资源独占**：每个资源只能给一个任务使用
 - **不可抢占**：资源只能由占有者自愿释放
 - **请求与保持**：每个任务请求资源，同时也占有资源
 - **循环等待**：存在一个任务对资源的等待环路
- **死锁预防**：破坏四个条件之一 → 调度必须考虑
 - **重要方法**：银行家算法
- **死锁解除**：消除四个条件之一

优先级反转现象

- 优先级反转 (priority inversion)

高优先级任务需要等待低优先级任务释放资源，而低优先级任务又正在等待中等优先级任务执行的现象。

优先级反转：描述

- **假设：**三个任务和优先级 $\text{Task}_1 > \text{Task}_2 > \text{Task}_3$
- **初始状态：**
 - **时刻t1：** Task_1 和 Task_2 等待事件挂起， Task_3 执行，占有资源S
 - **时刻t2：** Task_1 就绪，抢占 Task_3 执行， Task_3 阻塞
 - **时刻t3：** Task_1 申请资源S失败挂起， Task_3 重新执行
 - **时刻t4：** Task_2 就绪，抢占 Task_3 执行， Task_3 阻塞
 - **时刻t5：** Task_2 执行完， Task_3 重新执行
 - **时刻t6：** Task_3 执行完， Task_1 重新执行
- **结果：**最高优先级任务 Task_1 最后执行完成 → 优先级反转

优先级继承

优先级继承

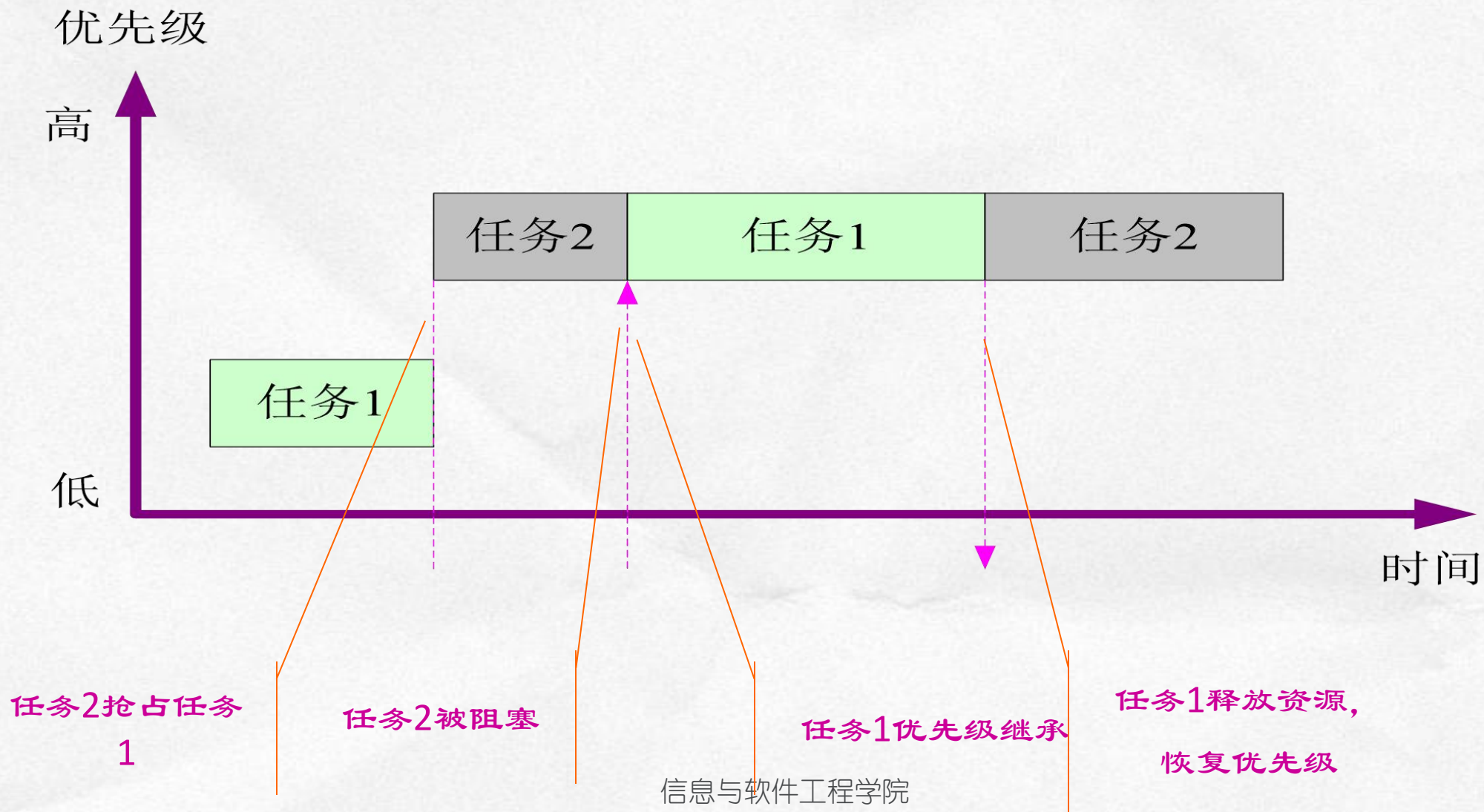
- 优先级继承协议

当一个任务在其使用的临界区阻塞了一个或多个高优先级任务时，该任务将不使用其原来的优先级，而使用被该任务所阻塞的所有任务的最高优先级作为其执行临界区的优先级。当该任务退出临界区时，又恢复到其最初的优先级。

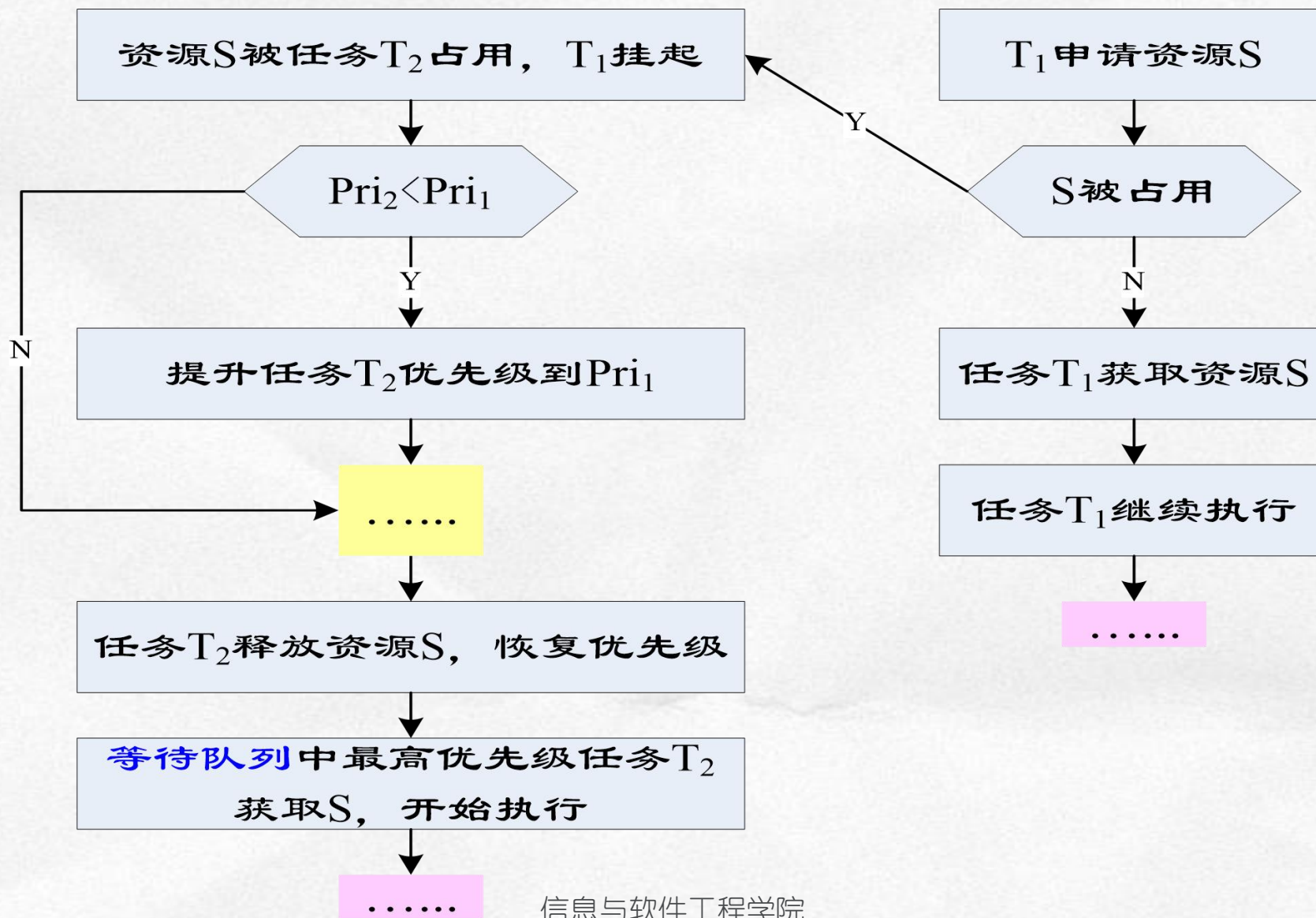
优先级继承：规则

协议规则号	描述
1	如果资源R在使用，则任务T被阻塞；如果资源R是空闲的，则资源R被分配给任务T
2	当较高优先级的任务T'，请求资源R时，任务T的优先级被提升到T'的优先级等级
3	当任务T释放资源R后，返回到先前的优先级

优先级继承：示意



优先级继承：基本处理流程



优先级继承：说明

- **继承性**：任务T进入临界区而阻塞了更高优先级的任务，则任务T将继承被阻塞任务的优先级，直到任务T退出临界区
- **动态性**：一个不相关的较高优先级任务仍可进行任务抢占，这是基于优先级可抢占调度模式的本性
- **传递性**：任务优先级在反转期间，被提升优先级的任务的优先级可以继续被提升
- **有界性**：任务的阻塞时间是有界的，但可能出现阻塞链，从而会加长阻塞时间，甚至造成死锁

优先级继承：优缺点

- **优点**：可以有效降低单个低优先级任务的阻塞时间
- **缺点**：高优先级任务可能被中优先级任务阻塞
- **缺点**：若一个高优先级任务所需多个临界资源分别被不同的低优先级任务占有，则阻塞可能相当长
- **缺点**：当有多个临界资源被需要时，不能保证避免死锁发生

优先级天花板

优先级天花板

- 优先级天花板协议

示例：

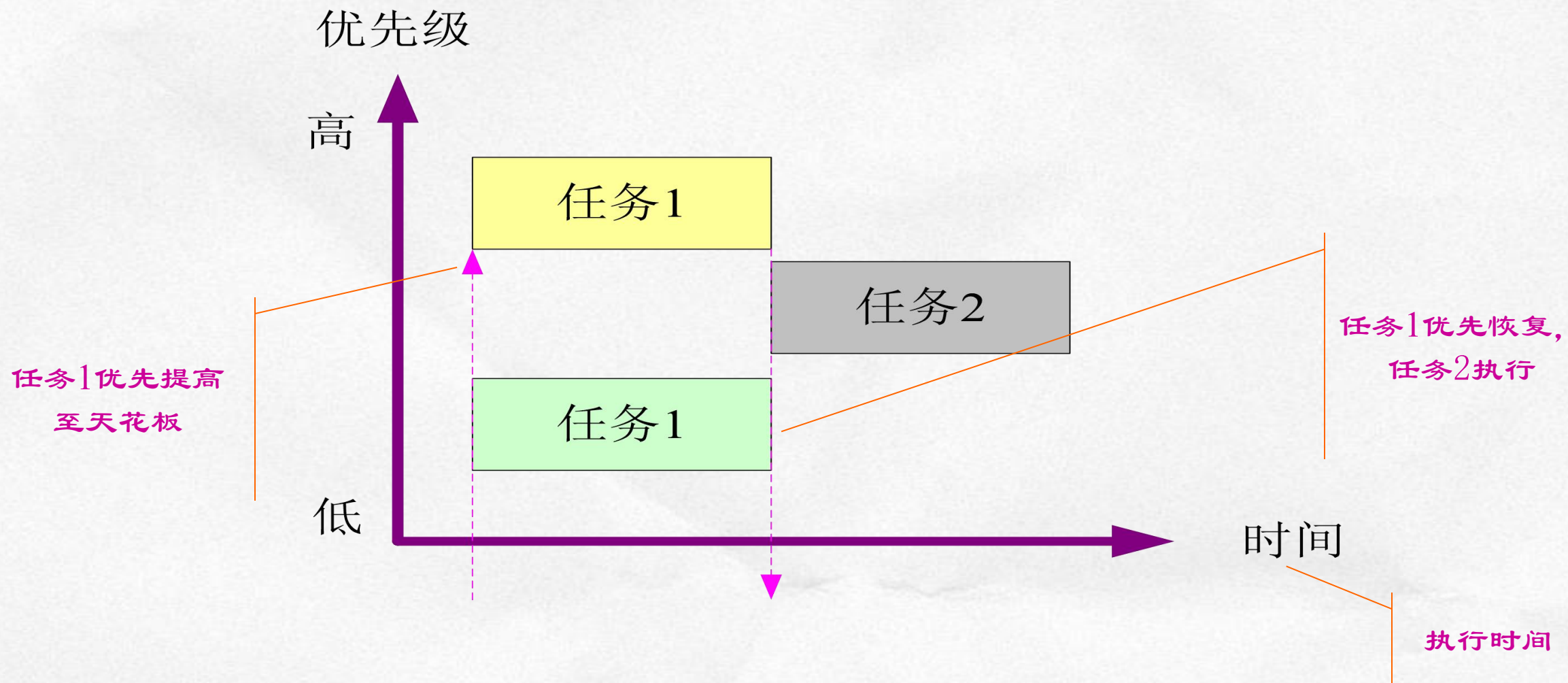
系统中有3个资源正在使用，资源R1的优先级天花板为4，资源R2的优先级天花板为6，资源R3的优先级天花板为9，则系统当前的优先级天花板为9。

所获得信号量的优先级天花板。

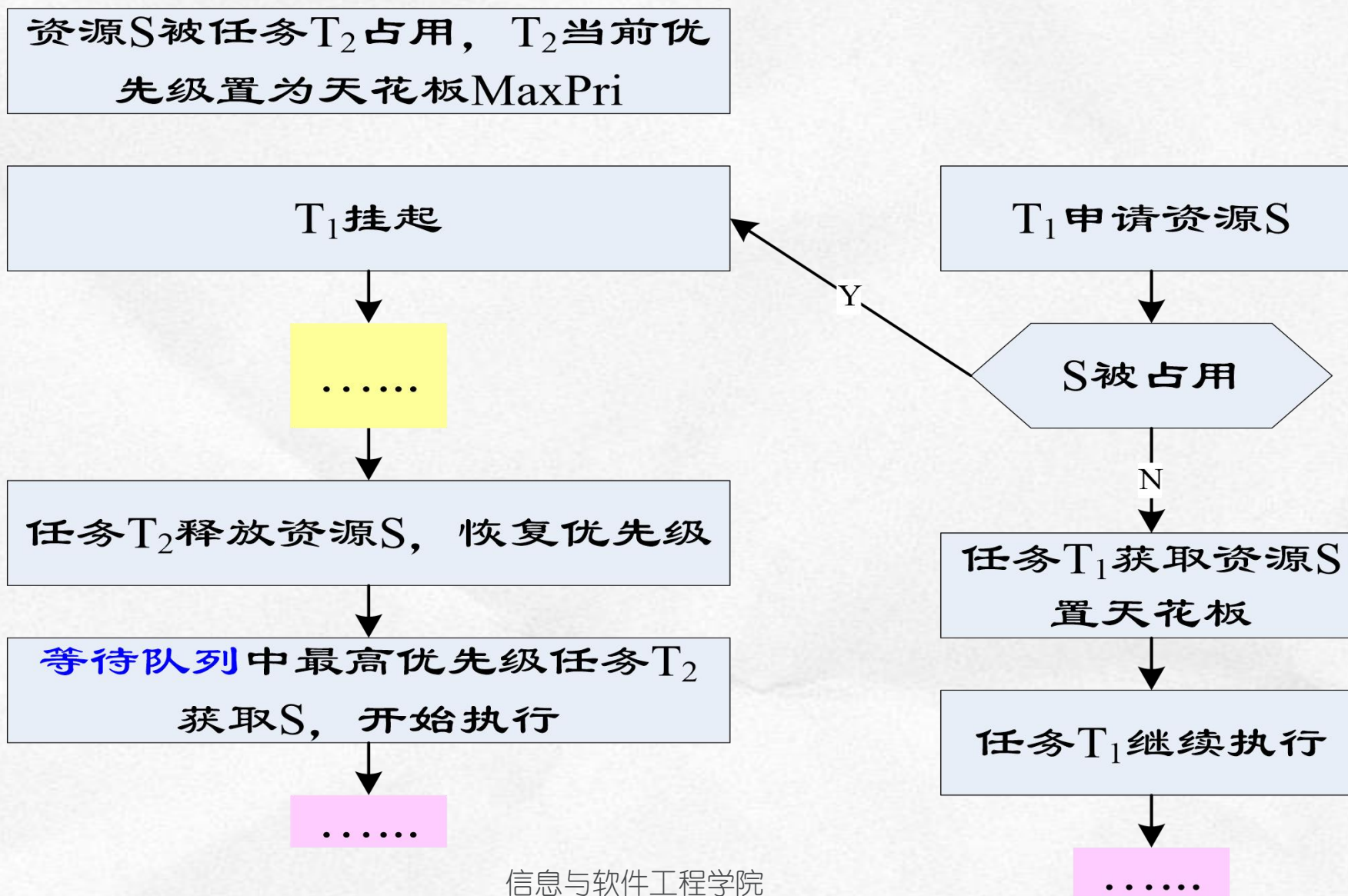
优先级天花板：规则

协议规则号	描述
1	如果资源R在使用，则任务T被阻塞
2	如果资源R是空闲的，则资源R被分配给任务T。 如果资源R的优先级天花板比任务T的优先级高， 则任务T的优先级被提升到资源R的优先级天花板等级。
3	在任意给定时间，任务T的执行优先级等于所有它占有的资源中最高优先级天花板
4	具有最高优先级天花板的资源被释放时，任务T的优先级更新为所占有资源的最高优先级天花板

优先级天花板：示意



优先级天花板：基本处理流程



优先级天花板：优缺点

- **优点**：可以大大降低高优先级任务的被阻塞时间
- **优点**：避免优先级逆转造成的死锁问题
- **缺点**：将阻塞不竞争资源的中间优先级任务

优先级继承和优先级天花板：比较

- **相同点** 都能解决优先级反转问题
- **执行效率** 继承协议可能多次改变占有临界资源的任务的优先级，而天花板协议只需改变一次：**后者效率高**
- **死锁问题** 优先级继承协议具有死锁和阻塞链问题，而优先级天花板协议可以解决这些问题
- **运行流程影响** 天花板协议可能影响某些中间优先级任务的完成时间；继承协议对任务执行流程的影响相对较小



谢谢！